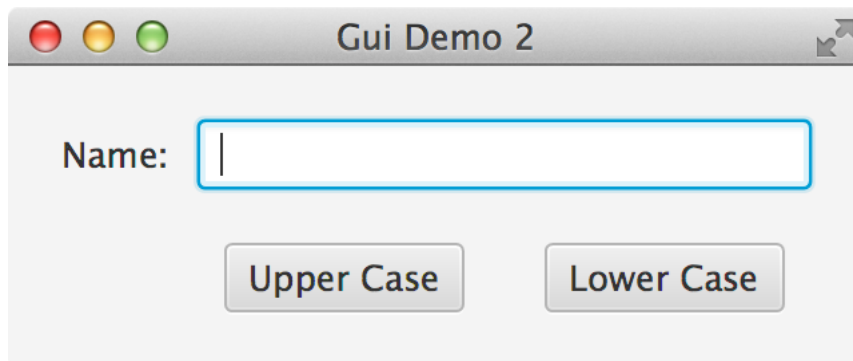


GRAPHICAL USER INTERFACE WITH JAVAFX – PART 1

The type of user interface covered in this note is a *graphical user interface* (GUI). In contrast, a user interface that uses `System.in` for input and `System.out` for output is a *console user interface* (or a *non-graphical user interface*).

An application with a GUI interface may look like this:



In Java, GUI based programs are implemented by using the classes from the *javafx* packages. These packages contain a wealth of ready-made classes for creating the user interface of GUI based programs. Examples are classes for windows, buttons, labels, and text fields.

We will collectively refer to these classes as the *JavaFX classes*.

Event-driven programs

A program with a GUI interface is only doing something when the user interacts with the program, e.g. by clicking a button. We say that the user has made an action that causes the program to *raise an event*, and the program must react on that event.

The program flow of a GUI based program is *event driven* (and the programming style of a GUI based program is *event driven programming*). In an event-driven program, the computer just waits most of the time for the user to interact with a component in the GUI. In many cases, when the user interacts with a GUI component, it raises an event. Examples of actions that raises an event: the user moves the cursor, the user clicks a button, or the user selects a menu choice.

For the program shown in the picture above, an event named an *action* event is raised when the user clicks one of the buttons. The programmer must write some code that executes when the user clicks the "All Upper Case" button, and some code that executes when the user clicks the "All Lower Case" button.

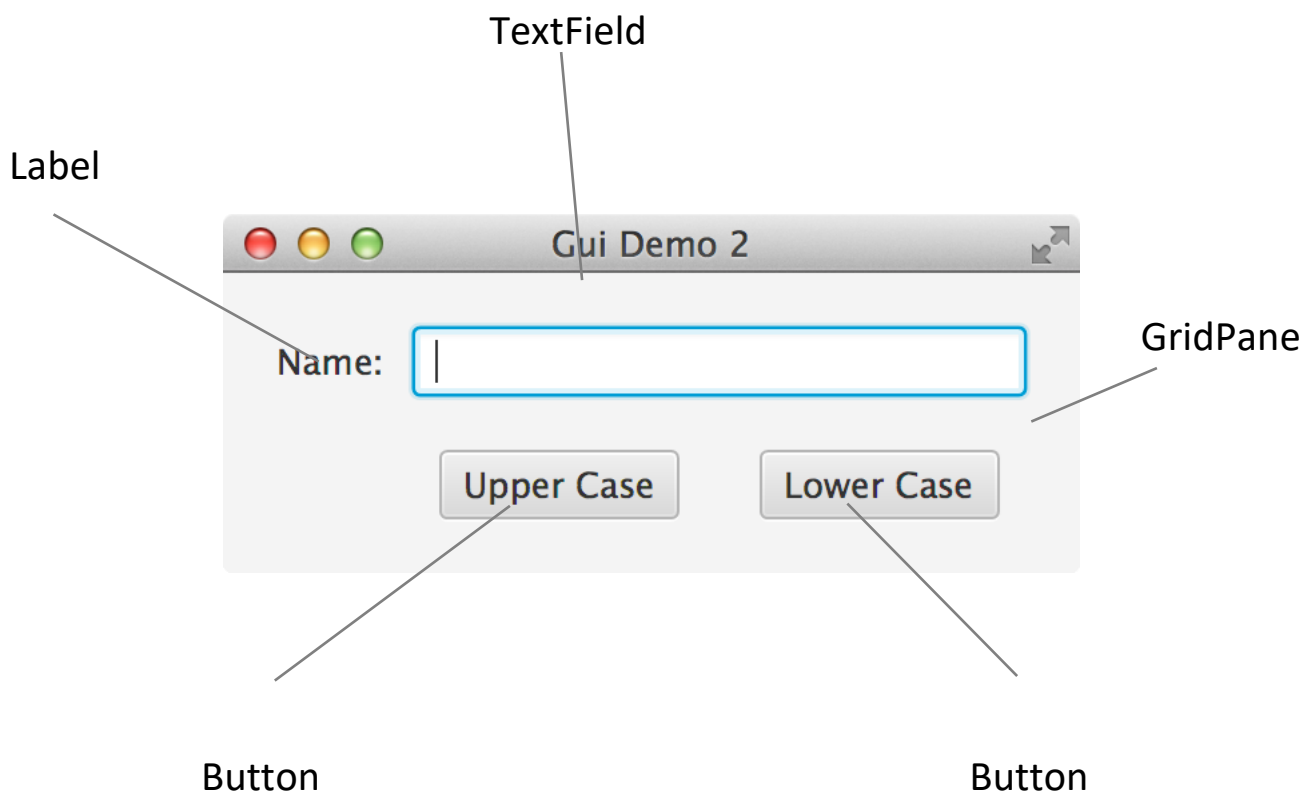
Event handling code is written in *event handling methods*, which executes when an event occurs. The event-handling methods are often named with the *Action* word, e.g. `upperCaseAction()`. This method must be connected to the button using the `setOnAction()` method (a method on the `Button` class).

To program a GUI based program, you will have to

- Create a graphical user interface with GUI components.
- Write event handling methods, like `upperCaseAction()`.
- Call the `setOnAction()` method to connect the button to the event handling method.

JavaFX components

To program with JavaFX, you will have to learn to use predefined components. In the example shown below we use a Label, a TextField and two Buttons placed inside a GridPane that covers the whole inner part of the window:



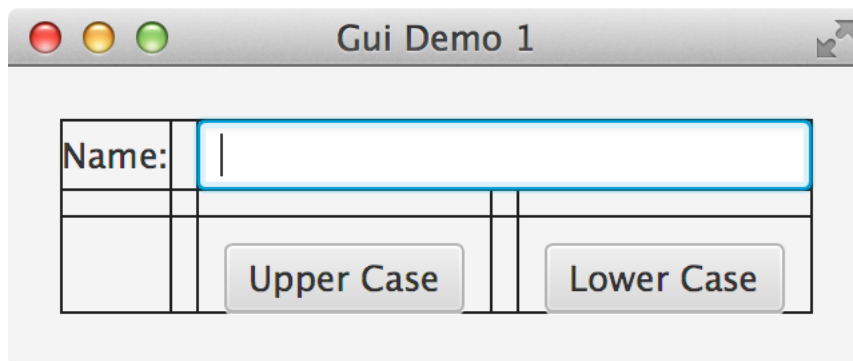
The frame (or border) of the window is drawn by the operating system (Windows, Mac OS X, etc.). A JavaFX object of type `GridPane` draws the inner content of the window above (there are other `Pane` types in addition to `GridPane`).

Demo 1: Create the visual GUI

We will look at a program that displays a window showing a label, a TextField and two buttons.

First, we will create the visual GUI.

The grid lines in the picture below shows columns and rows with gaps, and are often enabled by the developer when the GUI is under development.



The application consists of two classes named Gui and App. The code below creates a window containing one component: a GridPane. The grid pane is a container for other components.

The developer adds components to a grid pane by placing them in rows and columns. To make a better visual impression, padding and gaps between components are set in the grid pane.

Consider the class Gui below which is a simple JavaFX app drawing the window shown above.

```
package example;
```

```
// imports not shown here
```

```
public class Gui extends Application {
```

```
    @Override
```

```
    public void start(Stage stage) {  
        stage.setTitle("Gui Demo 1");  
        GridPane pane = new GridPane();  
        this.initContent(pane);
```

```
        Scene scene = new Scene(pane);  
        stage.setScene(scene);  
        stage.show();
```

```
    }
```

```
// -----
```

```
    private void initContent(GridPane pane) {  
        // enable this to show grid lines  
        pane.setGridLinesVisible(true);
```

```

// set padding of the pane
pane.setPadding(new Insets(20));
// set horizontal gap between components
pane.setHgap(10);
// set vertical gap between components
pane.setVgap(10);

// add a label to the pane (at col=0, row=0)
Label lblName = new Label("Name:");
pane.add(lblName, 0, 0);

// add a text field to the pane
// (at col=1, row=0, extending 2 cols and 1 row)
TextField txfName = new TextField();
pane.add(txfName, 1, 0, 2, 1);

// add a button to the pane (at col=1, row=1)
Button btnUpperCase = new Button("Upper Case");
pane.add(btnUpperCase, 1, 1);
GridPane.setMargin(btnUpperCase, new Insets(10, 10, 0, 10));

// add a button to the pane (at col=2, row=1)
Button btnLowerCase = new Button("Lower Case");
pane.add(btnLowerCase, 2, 1);
GridPane.setMargin(btnLowerCase, new Insets(10, 10, 0, 10));
}
}

```

The class declaration

```

public class Gui extends Application { . . . }

```

specifies that the Gui class is a subclass of the Application class. This implies that the Gui class inherits all the methods that the Application class has (e.g. setTitle(), setScene()).

The code above the dashed line is the same in (almost) all GUI applications (except for the title of the window).

In the initContent() method, the components are created, properties of the components are set, and the components are added to the pane.

The class App starts the Gui class:

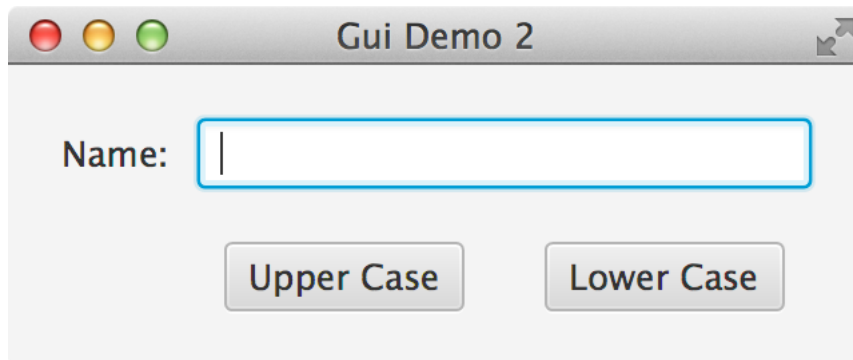
```

public class App {
    public static void main(String[] args) {
        Application.launch(Gui.class);
    }
}

```

Demo 2: Add event handling methods

Next, we will add code that makes the buttons work, that is, we want to code event-handling methods and connect the methods to the buttons.



New code is added to connect the button with event handling. The changes from the last example are marked below with yellow.

```
public class Gui extends Application {  
  
    // ...  
    // The code above this dashed line is exactly the same as in demo 1.  
    //-----  
  
    private final TextField txfName = new TextField();  
  
    private void initContent(GridPane pane) {  
        // show or hide grid lines  
        pane.setGridLinesVisible(true);  
  
        // set padding of the pane  
        pane.setPadding(new Insets(20));  
        // set horizontal gap between components  
        pane.setHgap(10);  
        // set vertical gap between components  
        pane.setVgap(10);  
  
        // add a label to the pane (at col=0, row=0)  
        Label lblName = new Label("Name:");  
        pane.add(lblName, 0, 0);  
  
        // add a text field to the pane  
        // (at col=1, row=0, extending 2 columns and 1 row)  
        // Deleted here is:  
        // TextField txfName = new TextField();  
        pane.add(txfName, 1, 0, 2, 1);  
    }  
}
```

```

// add a button to the pane (at col=1, row=1)
Button btnUpperCase = new Button("Upper Case");
pane.add(btnUpperCase, 1, 1);
GridPane.setMargin(btnUpperCase, new Insets(10, 10, 0, 10));

// connect a method to the button
btnUpperCase.setOnAction(event -> this.upperCaseAction());

// add a button to the pane (at col=2, row=1)
Button btnLowerCase = new Button("Lower Case");
pane.add(btnLowerCase, 2, 1);
GridPane.setMargin(btnLowerCase, new Insets(10, 10, 0, 10));

// connect a method to the button
btnLowerCase.setOnAction(event -> this.lowerCaseAction());
}

//-----
// Button actions

private void upperCaseAction() {
    String name = txtName.getText().trim();
    txtName.setText(name.toUpperCase());
}

private void lowerCaseAction() {
    String name = txtName.getText().trim();
    txtName.setText(name.toLowerCase());
}
}

```

The line

```
btnLowerCase.setOnAction(event -> this.lowerCaseAction());
```

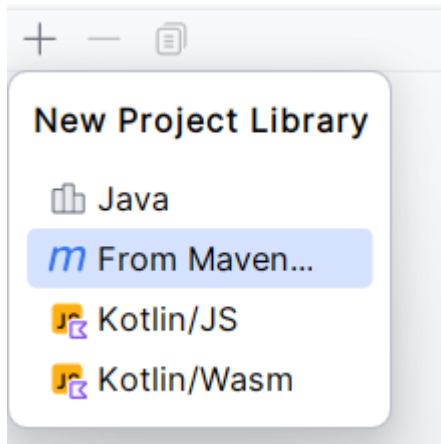
connects the `lowerCaseAction()` event handling method to the `btnLowerCase` button. *When the button is clicked, the `lowerCaseAction()` method will be executed.*

Please note that the `txtName` variable has changed from a local variable to a field. This is necessary because the variable is used in more than one method.

The demo project can be found on [benn-christensen/JavaFXDemo \(github.com\)](https://github.com/benn-christensen/JavaFXDemo)

How-to: Steps to create a simple JavaFX application

1. Create a new IntelliJ project, and use one of the examples in the previous section as starting point for a new GUI application.
2. Under “Project Structure” -> “Libraries” -> press the + sign -> “From Maven”



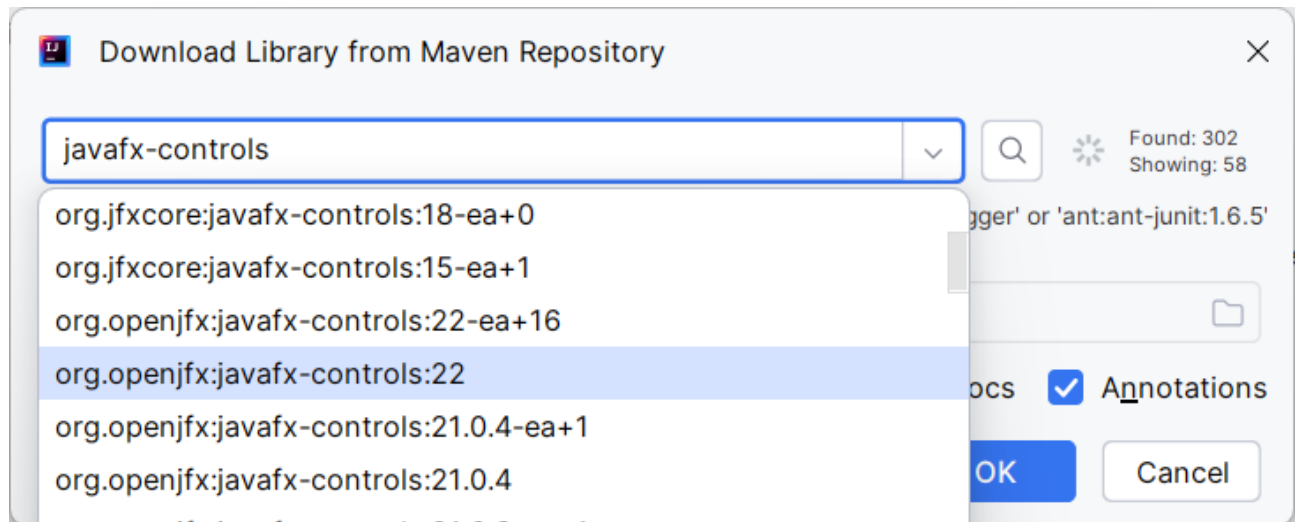
Search for “javafx-controls”



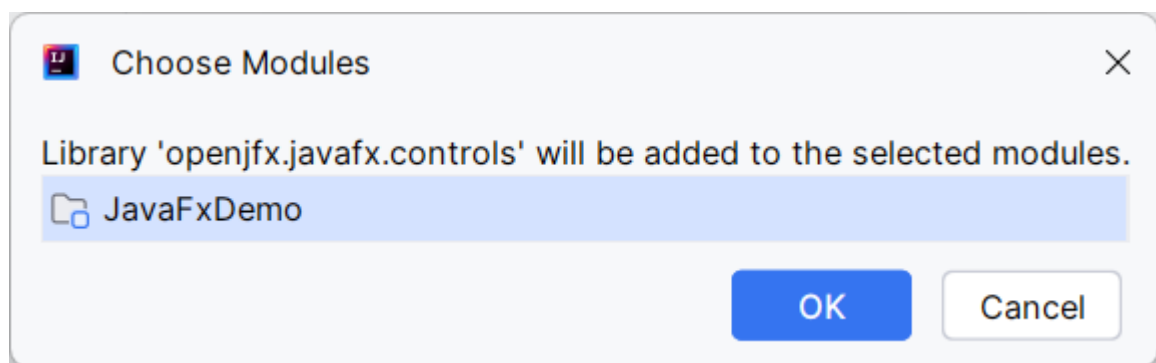
Choose “org.openjfx:javafx-controls:22”

Mark “Transitive dependencies”, “Sources”, “Javadocs” and “Annotations”

And press “OK”



Press “OK” to finish the import.



3. For each component that needs to be in the window:
 - Create the component, and assign it to a variable. The variable is either a local variable or a field (use a field if the variable needs to be accessible from more than one method).
 - Add the component to the pane.
 - Set properties for the component.
4. Set action events for buttons:
 - Add event handler methods to the Gui class. The methods must contain code to execute when the buttons are pressed.
 - Connect the methods to the buttons.

TextArea

TextArea is a component that can contain text consisting of many lines. Scrollbars are automatically shown if needed.

You can use a TextArea to show output from your program. Here is an example with a non-editable TextArea. In the code below the parts concerning the text area are marked with yellow.

```
public class Gui extends Application {

    @Override
    public void start(Stage stage) {
        stage.setTitle("Gui Demo TextArea");
        GridPane pane = new GridPane();
        this.initContent(pane);

        Scene scene = new Scene(pane);
        stage.setScene(scene);
        stage.show();
    }

    // -----

    private final TextArea txaDescription = new TextArea();

    private void initContent(GridPane pane) {
        pane.setGridLinesVisible(false);

        pane.setPadding(new Insets(20));
        pane.setHgap(20);
        pane.setVgap(10);

        Label lblName = new Label("TextArea:");
        pane.add(lblName, 0, 0);

        pane.add(txaDescription, 0, 1);
        txaDescription.setPrefRowCount(7);
        txaDescription.setPrefWidth(270);
        txaDescription.setEditable(false);

        Button btnFill = new Button("Fill TextArea");
        pane.add(btnFill, 1, 1);
        btnFill.setOnAction(event -> this.fillAction());
    }

    private void fillAction() {
        txaDescription.setText(this.getDescription());
    }

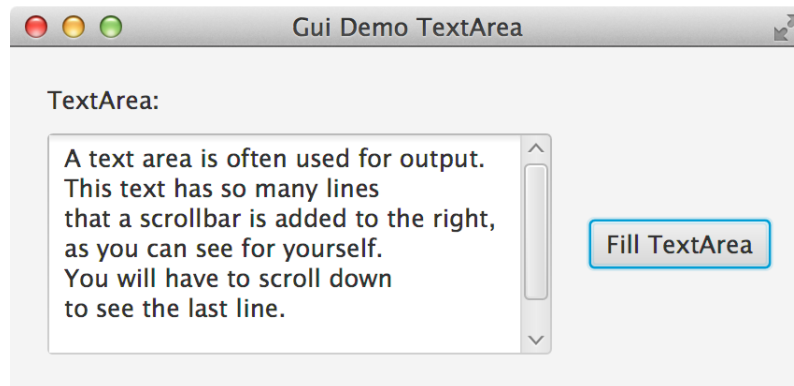
    private String getDescription() {
        StringBuilder sb = new StringBuilder();
        sb.append("A text area is often used for output.\n");
        sb.append("This text has so many lines\n");
        sb.append("that a scrollbar is added to the right, \n");
        sb.append("as you can see for yourself.\n");
        sb.append("You will have to scroll down\n");
        sb.append("to see the last line.\n");
    }
}
```

```

        sb.append("\n\n\n\n\n\n");
        sb.append("This is the last line.");
        return sb.toString();
    }
}

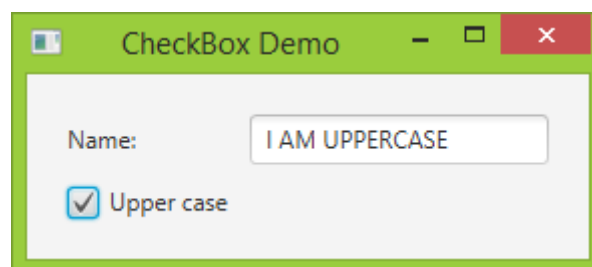
```

The program's GUI will look like this (on a Mac):



CheckBox

A checkbox is a bit like a button, except that it has two states. It can be checked or unchecked. It is useful if you want to indicate if something is true or not. Look at the example below (it is in the *DemoCheckBox* package).



CheckBoxes are about as easy to make as buttons. This is one way to do it:

```

// CheckBox to indicate upper / lower case
CheckBox checkBox = new CheckBox();
checkBox.setText("Upper case");
pane.add(checkBox, 0, 1);

checkBox.setOnAction(event -> {
    if (checkBox.isSelected()) {
        upperCaseAction();
    } else {

```

```
        lowerCaseAction();  
    }  
});
```

Here, we are using the lambda notation in a slightly different way. After the “->” arrow notation, we create a block, and inside this block, we have an if-else statement that checks, whether the checkbox is selected or not. Depending on the state of the check box, one of the two methods `upperCaseAction()` and `lowerCaseAction()` is called. Try it out!